

Self-Learning: Merge Sort & Quick Sort — Data Structures

Definition: **Merge Sort** and **Quick Sort** are high-performance $O(N \log N)$ sorting algorithms based on the **Divide and Conquer** paradigm.

1. Detailed Technical Explanation

1. Merge Sort

- **Concept:** Divides array into two halves, recursively sorts both halves, and merges the two sorted halves into a single sorted array.
- **Recurrence Relation:** $T(N) = 2T(N/2) + \Theta(N)$ implies $O(N \log N)$ in all cases.
- **Key Trait:** **Stable** sort with guaranteed $O(N \log N)$ time, but requires $O(N)$ auxiliary memory.

```
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

2. Quick Sort

- **Concept:** Selects a **Pivot** element, partitions the array such that all elements smaller than the pivot are on the left and all larger elements are on the right, then recursively sorts the left and right partitions.
- **Key Trait:** **In-Place** sort with fast practical performance ($O(N \log N)$ average), but worst-case time is $O(N^2)$ (when already sorted and choosing boundary pivot).

```

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Lomuto Partition scheme
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
    }
    int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

2. Comparison: Merge Sort vs Quick Sort

Feature	Merge Sort	Quick Sort
Best-Case Time	$O(N \log N)$	$O(N \log N)$
Average-Case Time	$O(N \log N)$	$O(N \log N)$
Worst-Case Time	$O(N \log N)$ (Guaranteed)	$O(N^2)$ (Mitigated via Randomized Pivot)
Auxiliary Space	$O(N)$ (Temporary arrays)	$O(\log N)$ (In-Place stack space)
Stability	Stable	Unstable

3. Quick Recall Flow

Merge Sort: Divide in half -> Recursively Sort -> Merge sorted halves ($O(N \log N)$ Stable, $O(N)$ space)
 Quick Sort: Choose Pivot -> Partition Left < Pivot < Right -> Recursively Sort ($O(N \log N)$ In-Place)